New Relic APM — Day 4 Lab Guide

Hands-on labs for *Custom instrumentation, RUM and synthetics* · Spring PetClinic and easyTravel on Linux · New Relic Java agent, browser agent and synthetics job manager

How to use this guide

Each lab matches a module of the New Relic APM — Day 4 Study Guide: read the module, then do its lab. The labs build on the Day 1–3 lab system.

Lab	Title	Module	Time
4.1	Instrument the owner search: custom methods, attributes, events and naming	Module 10	60 min
4.2	Enable RUM through the APM agent and read the user experience	Module 11	60 min
4.3	Synthetic monitoring from a private location, with SLA and alerts	Module 12	75 min

Before you start

- Day 3 is complete: `pcdc` works, PetClinic answers on port 8090 with the PostgreSQL data from Lab 3.2, the pattern index exists, and the infrastructure agent is running.
- The PetClinic source is in `~/nr-lab/petclinic-src` (Lab 3.2, Step 1). Today you change it, so commit nothing from it to a shared repository.
- The VM needs about **4 GB more memory and 10 GB more disk** for today's additions (headless browsers, the synthetics job manager and its runtime images). Check with `free -h` and `df -h ~`. Stop easyTravel's load generator if memory is short: `etdc stop headless-customer-loadgen`.

```
cd ~/nr-lab
pcdc ps --format "table {{.Name}}\t{{.Status}}"
curl -s -o /dev/null -w "petclinic: HTTP %{http_code}\n" http://localhost:8090/
free -h | head -2; df -h ~ | tail -1
```

Conventions

- `$` lines run on the lab VM as `trainee`, from `~/nr-lab` unless a step says otherwise.
- **Expected output** pictures show terminal output; **Expected view** pictures are redrawn New Relic screens — your numbers, names and layout will differ.
- Every PetClinic rebuild uses the same command. Define it once:

```
cat >> ~/.bashrc <<'EOF'
pcbuild() {
  NR_VERSION=$(python3 -c "import zipfile,os; m=zipfile.ZipFile(os.path.expanduser('~/nr-
lab/agent/petclinic/newrelic.jar')).read('META-INF/MANIFEST.MF').decode(); print([l.split(':
')[1].strip() for l in m.splitlines() if l.startswith('Implementation-Version')][0])")
  echo "building against newrelic-api $NR_VERSION"
  docker run --rm -v "$HOME/nr-lab/petclinic-src":/src -v "$HOME/.m2":/root/.m2 -w /src
maven:3.9-eclipse-temurin-17 \
    mvn -q -DskipTests -Dspring-javaformat.skip=true -Dcheckstyle.skip -
Dnewrelic.version="$NR_VERSION" package \
  && cp ~/nr-lab/petclinic-src/target/spring-petclinic-*.jar ~/nr-lab/petclinic.jar \
  && pcdc restart petclinic && sleep 60 \
```

```
  && curl -s -o /dev/null -w "petclinic: HTTP %{http_code}\n" http://localhost:8090/
}
EOF
source ~/.bashrc
```

The API jar version matches the agent version, read from the agent's manifest. The formatting and style checks are skipped because the lab's edits are not formatted to the project's rules.

Lab 4.1 — Instrument the owner search: custom methods, attributes, events and naming

Goal: the owner search is PetClinic's critical transaction, but New Relic sees only "a Spring controller that calls the database". Add a custom method with no code change, then use the Java agent API to add business context — attributes, a custom event, and transaction names that separate searches that find nothing, one owner or many.

Time: 60 minutes. **Module:** 10.

Step 1 — Look at what the agent sees today

Open **APM & Services** → **petclinic** → **Transactions** → `owners (GET)` → **a transaction trace** (or Distributed tracing, filtered on that name). Note the segments: the Spring controller, then the database calls directly under it. The private method that builds the page of results — `OwnerController.findPaginatedForOwnersLastName` — is not visible.

Step 2 — A custom method without changing code (XML)

```
mkdir -p ~/nr-lab/agent/petclinic/extensions
cat > ~/nr-lab/agent/petclinic/extensions/petclinic-owner-search.xml <<'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<extension xmlns="https://newrelic.com/docs/java/xsd/v1.0"
           name="petclinic-owner-search" version="1.0" enabled="true">
  <instrumentation metricPrefix="Custom">
    <pointcut transactionStartPoint="false">
      <className>org.springframework.samples.petclinic.owner.OwnerController</className>
      <method>
        <name>findPaginatedForOwnersLastName</name>
      </method>
    </pointcut>
  </instrumentation>
</extension>
EOF
pcdc restart petclinic && sleep 60
grep -i "petclinic-owner-search" ~/nr-lab/agent/petclinic/logs/newrelic_agent.log | tail -2
```

Lab 4.1 · Step 2 · XML custom instrumentation — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ mkdir -p ~/nr-lab/agent/petclinic/extensions
trainee@nr-lab:~/nr-lab$ cat > ~/nr-lab/agent/petclinic/extensions/petclinic-owner-search.xml
                           <<'EOF'
(extension petclinic-owner-search: OwnerController.findPaginatedForOwnersLastName)
trainee@nr-lab:~/nr-lab$ ls ~/nr-lab/agent/petclinic/extensions
petclinic-owner-search.xml
trainee@nr-lab:~/nr-lab$ pcdc restart petclinic && sleep 60
[+] Restarting 1/1
 ✔ Container petclinic  Started
trainee@nr-lab:~/nr-lab$ grep -i "petclinic-owner-search"
                           ~/nr-lab/agent/petclinic/logs/newrelic_agent.log | tail -2
2026-10-29T11:02:14 [1] INFO: Reading custom extension file
    /opt/newrelic/extensions/petclinic-owner-search.xml
2026-10-29T11:02:14 [1] INFO: Loaded extension petclinic-owner-search version 1.0
the exact log wording varies between agent versions
```

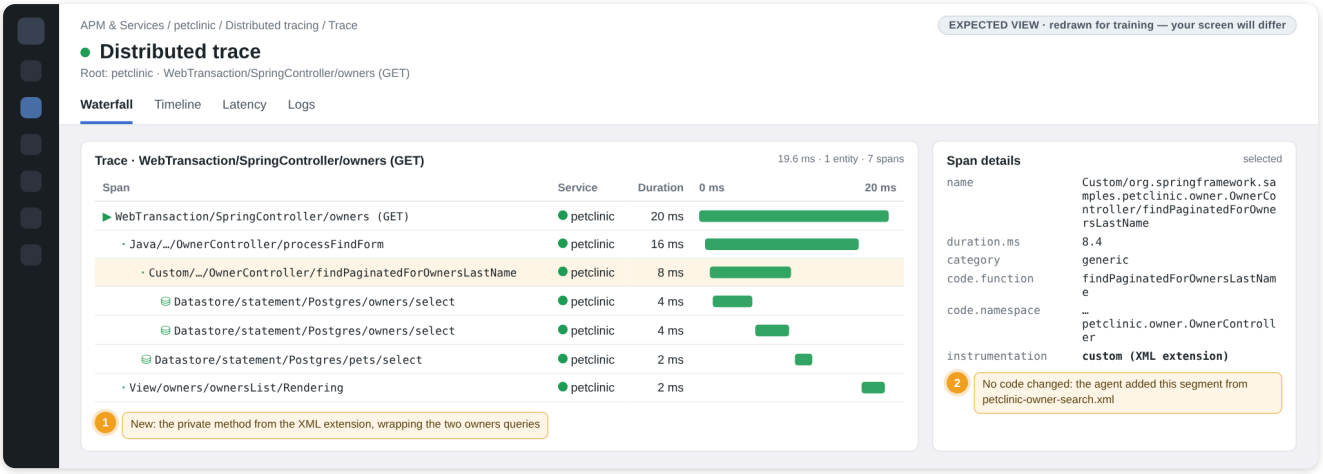
Expected output — the XML extension in the agent's extensions folder, loaded after a restart

The agent reads `extensions/` next to `newrelic.jar`. The restart makes sure the class is instrumented even though it was already loaded. The same file can be uploaded in **petclinic → Settings → Instrumentation**.

Step 3 — See the new segment

After a few minutes of traffic (`pcload.sh` from Day 3 is still running), open a new trace of the owner search:

- A segment named like `OwnerController.findPaginatedForOwnersLastName` now sits between the controller and the two `owners` queries.
- Its time is now separated from the controller's own time.



Expected view — the owner search trace with the new custom segment around the two owners queries

```
SELECT average(newrelic.timeslice.value) * 1000 AS 'ms' FROM Metric
WHERE appName = 'petclinic' AND metricTimesliceName LIKE 'Custom/%'
FACET metricTimesliceName TIMESERIES SINCE 30 minutes ago
```

Step 4 — Add business context with the agent API

Add the API dependency, a small telemetry class and one call in the controller:

```
cd ~/nr-lab/petclinic-src

# 1. The API dependency (version passed by pcbuild)
sed -i '0,/<dependencies>/s||<dependencies>\n    <dependency>\n
<groupId>com.newrelic.agent.java</groupId>\n      <artifactId>newrelic-api</artifactId>\n
<version>${newrelic.version}</version>\n    </dependency>|' pom.xml

# 2. The telemetry class
cat > src/main/java/org/springframework/samples/petclinic/owner/OwnerSearchTelemetry.java
<<'EOF'
package org.springframework.samples.petclinic.owner;

import java.util.HashMap;
import java.util.Map;

import com.newrelic.api.agent.NewRelic;
import com.newrelic.api.agent.Trace;

import org.springframework.data.domain.Page;

/**
```

```
 * Lab 4.1: business context for the owner search. No personal data: the last name
 * itself is never sent, only its length and what the search found.
 */
final class OwnerSearchTelemetry {

    private OwnerSearchTelemetry() {
    }

    @Trace(metricName = "Custom/OwnerSearch/record")
    static void record(String lastName, int page, Page<Owner> results) {
        long total = results.getTotalElements();
        String outcome = (total == 0) ? "none" : (total == 1) ? "single" : "many";

        // 1. Custom attributes on the transaction (and, in Lab 4.2, on its page view)
        NewRelic.addCustomParameter("search.outcome", outcome);
        NewRelic.addCustomParameter("search.resultCount", total);
        NewRelic.addCustomParameter("search.prefixLength", lastName.length());

        // 2. An attribute on this method's span only
        NewRelic.getAgent().getTracedMethod().addCustomAttribute("search.page", page);

        // 3. Segment the transaction by outcome: three names, never one per last name
        NewRelic.setTransactionName("OwnerSearch", "/" + outcome);

        // 4. One custom event per search
        Map<String, Object> event = new HashMap<>();
        event.put("outcome", outcome);
        event.put("resultCount", total);
        event.put("prefixLength", lastName.length());
        event.put("page", page);
        NewRelic.getAgent().getInsights().recordCustomEvent("OwnerSearch", event);
    }

}
EOF

# 3. Call it in processFindForm, right after the search runs
sed -i '/Page<Owner> ownersResults = findPaginatedForOwnersLastName(page, lastName);/a\
OwnerSearchTelemetry.record(lastName, page, ownersResults);' \
  src/main/java/org/springframework/samples/petclinic/owner/OwnerController.java
grep -n "OwnerSearchTelemetry"
src/main/java/org/springframework/samples/petclinic/owner/OwnerController.java
grep -A3 "newrelic-api" pom.xml
cd ~/nr-lab && pcbuild
```

Lab 4.1 · Step 4 · Agent API in PetClinic — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab/petclinic-src$ grep -n "OwnerSearchTelemetry" src/main/java/org/springfr
                                       amework/samples/petclinic/owner/OwnerController.java
108:OwnerSearchTelemetry.record(lastName, page, ownersResults);
trainee@nr-lab:~/nr-lab/petclinic-src$ grep -A3 "newrelic-api" pom.xml
      <artifactId>newrelic-api</artifactId>
      <version>${newrelic.version}</version>
    </dependency>
    <dependency>
trainee@nr-lab:~/nr-lab/petclinic-src$ cd ~/nr-lab && pcbuild
building against newrelic-api 8.25.0
(mvn -q prints nothing on success)
[+] Restarting 1/1
 ✔ Container petclinic  Started
petclinic: HTTP 200
the version printed is your agent's version; 8.25.0 is an example
```

Expected output — the dependency and the call added, PetClinic rebuilt against the agent's API version and restarted

Read the class before you build it: which values leave the application, and which do not? The last name a user types is personal data; its length and the outcome are not.

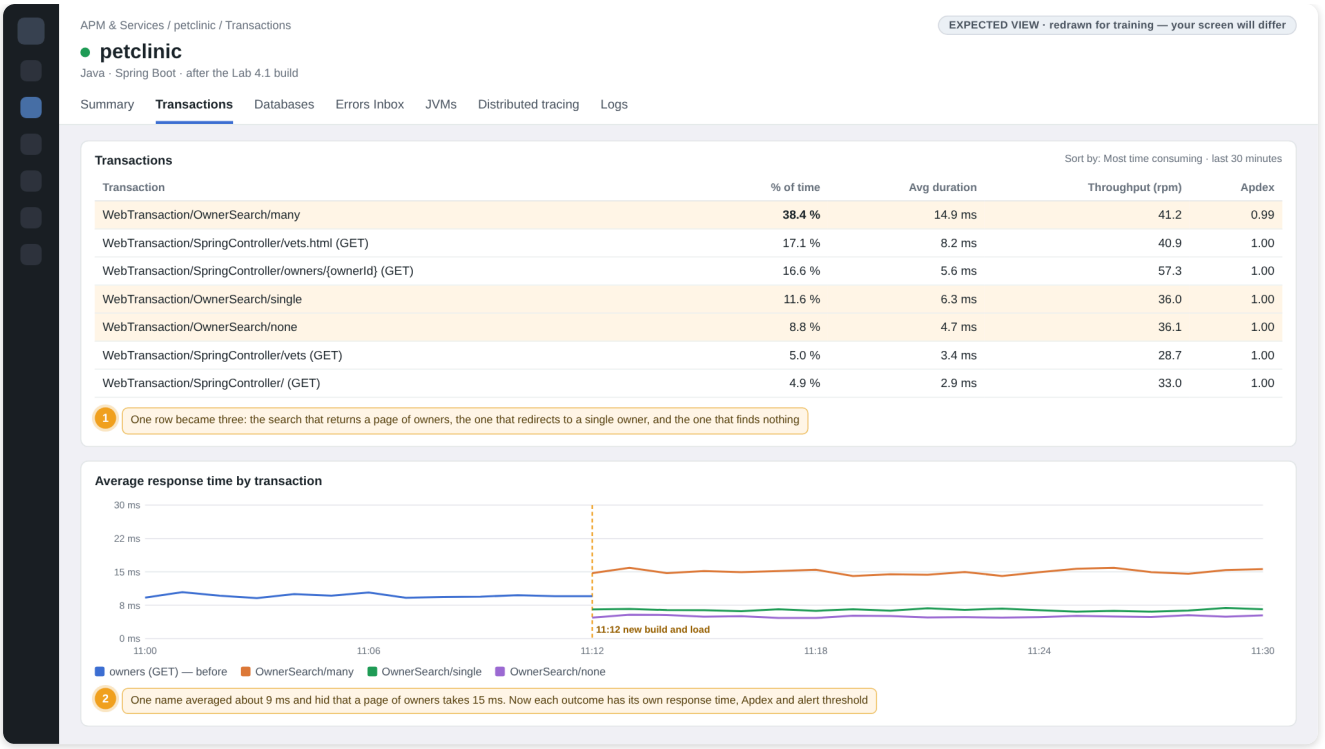
Step 5 — Traffic for every outcome

The Day 3 load script only searches for names that return many owners. Replace it with one that also finds one owner and none:

```
pkill -f pcload.sh
cat > ~/nr-lab/pcload.sh <<'EOF'
#!/usr/bin/env bash
# Owner searches with all three outcomes, owner pages and the vet list - about 4 requests per
second
while true; do
  curl -s -o /dev/null "http://localhost:8090/owners?lastName=Last$((RANDOM % 50000))"   #
many
  curl -s -o /dev/null "http://localhost:8090/owners?lastName=Franklin"                  #
single
  curl -s -o /dev/null "http://localhost:8090/owners?lastName=Nobody$RANDOM"             #
none
  curl -s -o /dev/null "http://localhost:8090/owners/$((RANDOM % 2000 + 1))"
  curl -s -o /dev/null "http://localhost:8090/vets.html"
  sleep 1
done
EOF
nohup ~/nr-lab/pcload.sh > /dev/null 2>&1 &
```

Step 6 — Segmented transaction names

After 5 minutes, open **petclinic** → **Transactions**. The single `owners (GET)` row is replaced by three names under the `OwnerSearch` category, each with its own response time, throughput and Apdex.



Expected view — the transaction list: the owner search now appears as three outcome-based names with different response times

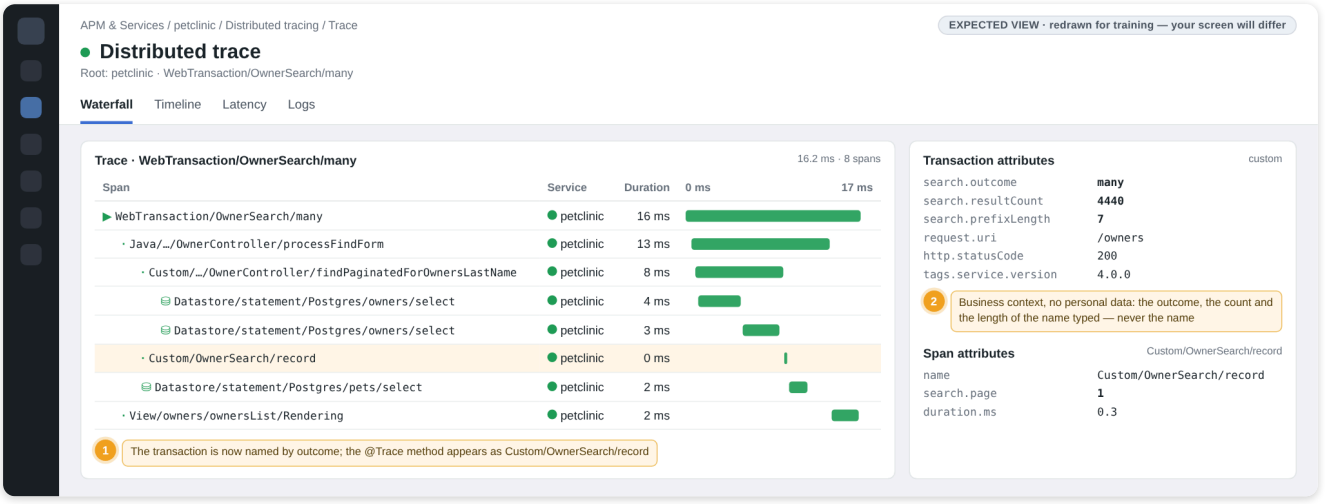
```
SELECT count(*), average(duration) * 1000 AS 'avg ms', percentile(duration, 95) * 1000 AS
'p95 ms'
FROM Transaction WHERE appName = 'petclinic' AND name LIKE '%OwnerSearch%'
FACET name SINCE 15 minutes ago
```

Question 1. Why is `"/" + outcome` a safe transaction name and `"/" + lastName` a dangerous one?

Step 7 — Attributes in traces

Open a trace of `OwnerSearch/many`:

- **Transaction attributes** (trace details or the attributes panel) include `search.outcome`, `search.resultCount` and `search.prefixLength`.
- The `Custom/OwnerSearch/record` segment carries the span attribute `search.page`.



Expected view — trace details: the custom segment and the custom attributes on the transaction and on the span

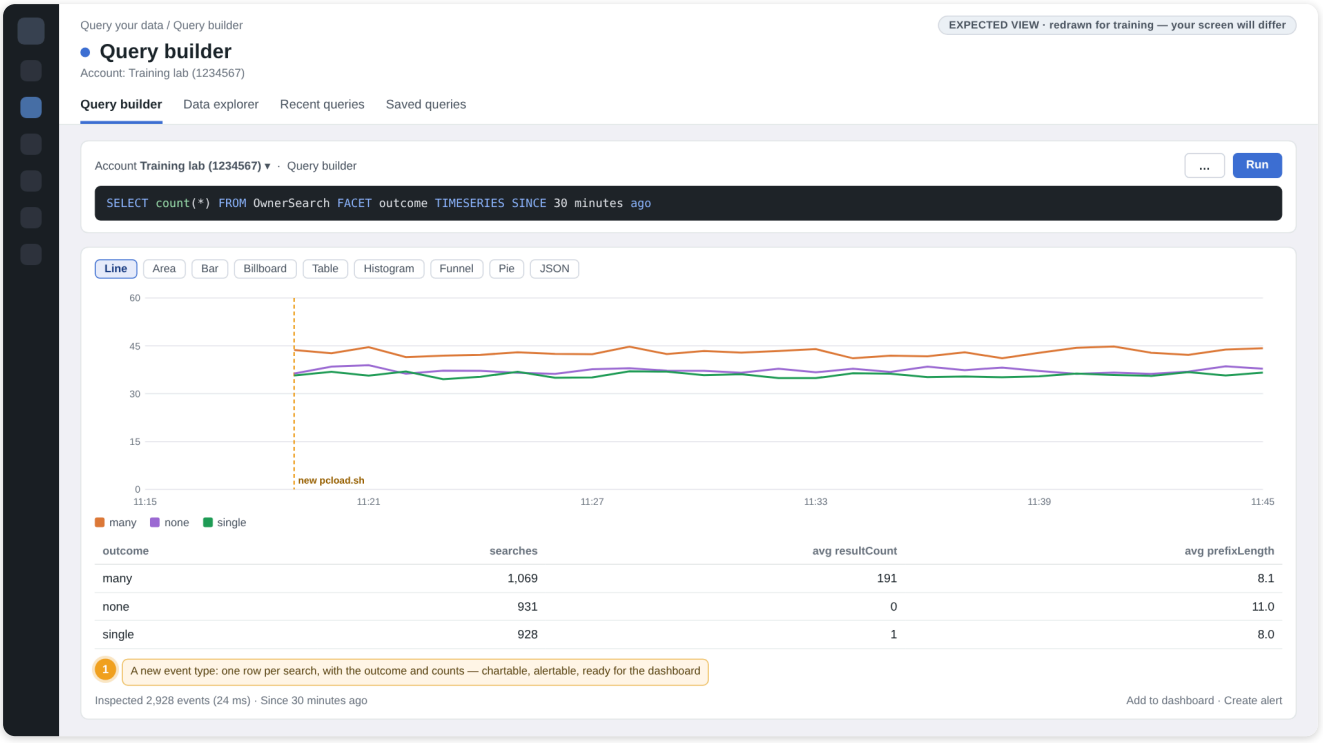
```
SELECT percentile(duration, 95) * 1000 AS 'p95 ms', count(*) FROM Transaction
WHERE appName = 'petclinic' FACET search.outcome SINCE 15 minutes ago

SELECT average(duration.ms), count(*) FROM Span
WHERE appName = 'petclinic' AND name LIKE 'Custom/OwnerSearch%' FACET search.page SINCE 15
minutes ago
```

Step 8 — The custom event

```
SELECT count(*) FROM OwnerSearch FACET outcome TIMESERIES SINCE 30 minutes ago

SELECT average(resultCount), max(resultCount), average(prefixLength) FROM OwnerSearch
WHERE outcome = 'many' SINCE 30 minutes ago
```



Expected view — the OwnerSearch custom event: searches per outcome over time

`OwnerSearch` is now an event type like `Transaction`: you can chart it, alert on it ("more than 20 % of searches find nothing") and put it on the Day 3 dashboard.

Step 9 — Check the cost

```
SELECT uniqueCount(name) AS 'transaction names' FROM Transaction WHERE appName = 'petclinic'
SINCE 1 hour ago
SELECT count(*) FROM OwnerSearch SINCE 1 hour ago
```

The number of transaction names stays small (the outcome has three values). The event count is one per search — fine here, but in a busy production service, check the agent's custom event limits before recording an event per request.

> The Day 3 dashboard and queries that use `name LIKE '%owners (GET)%'` no longer match the owner search. Change them to `name LIKE '%OwnerSearch%'` — every rename has this cost.

Check your work

- ☐ The custom segment from the XML extension appears in owner-search traces.
- ☐ Three `OwnerSearch` transaction names, the `search.*` attributes and the `OwnerSearch` event all appear.
- ☐ No last name, owner name or other personal data is sent in any attribute or event.

Answer. 1 — `outcome` has three possible values, so there are three names. `lastName` comes from user input: every different name typed becomes a new transaction name, which causes a metric grouping issue and also sends personal data.

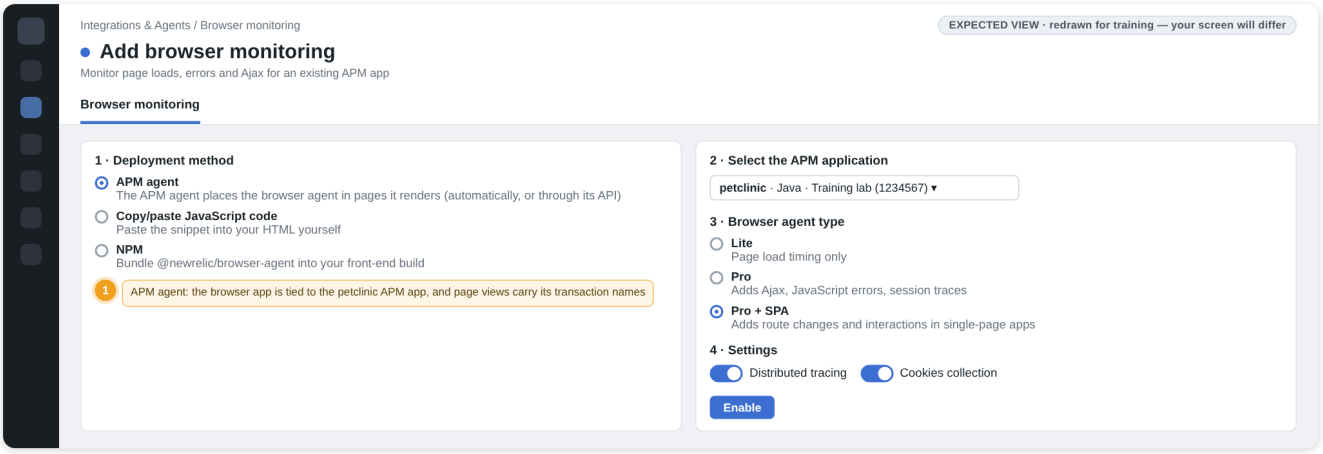
Lab 4.2 — Enable RUM through the APM agent and read the user experience

Goal: turn on browser monitoring for PetClinic through the Java agent, generate real browser traffic, plant a front-end bug and an Ajax call, and read the user's side of the application: page load phases, Core Web Vitals, JavaScript errors, browser-to-backend traces, session traces and geography.

Time: 60 minutes. **Module:** 11.

Step 1 — Enable browser monitoring for the APM app

1. **+ Integrations & Agents** (or **Add data**) → **Browser monitoring**.
2. Deployment method: **APM agent** (the agent places the script), not copy/paste.
3. Select the APM app **petclinic**; browser agent type **Pro + SPA**; leave distributed tracing **on**.
4. **Enable**. A browser app named `petclinic` appears under **Browser**.



Expected view — adding browser monitoring with the APM agent method for the petclinic APM app

Step 2 — Put the browser agent into PetClinic pages

The Java agent inserts the browser agent automatically only into compiled JSPs. PetClinic uses Thymeleaf, so the page template asks the agent for the script through the API. You also add two things to the vet list page for the next steps: an Ajax call and a deliberate JavaScript error.

```
cd ~/nr-lab/petclinic-src

# 1. Make the agent's browser header available to every template
cat > src/main/java/org/springframework/samples/petclinic/system/BrowserTimingAdvice.java
<<'EOF'
package org.springframework.samples.petclinic.system;

import com.newrelic.api.agent.NewRelic;

import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ModelAttribute;

/**
 * Lab 4.2: makes the New Relic browser agent available to every Thymeleaf page as
 * ${newrelic.browserHeader}. The header is generated when the page renders, after the
 * controller has named the transaction and added its attributes.
 */
@ControllerAdvice
```

```
public class BrowserTimingAdvice {

    @ModelAttribute("newrelic")
    public BrowserTimingAdvice newrelic() {
        return this;
    }

    public String getBrowserHeader() {
        return NewRelic.getBrowserTimingHeader();
    }

}
EOF

# 2. Output it near the top of <head>, after the meta tags, in the shared layout
sed -i '/<meta name="viewport"/a\  <th:block th:utext="${newrelic.browserHeader}">
</th:block>' \
  src/main/resources/templates/fragments/layout.html

# 3. On the vet list: an Ajax call to the JSON endpoint, and a bug on about half the visits
sed -i 's|</body>|<script>\n  fetch("/vets", { headers: { "Accept": "application/json" } })\n
.then(function (r) { return r.json(); })\n    .then(function (v) { document.title =
document.title + " (" + v.vetList.length + " vets)"; });\n  window.addEventListener("load",
function () {\n    if (Math.random() < 0.5) { petclinicUi.highlightVets(); }\n
});\n</script>\n</body>|' \
  src/main/resources/templates/vets/vetList.html

# 4. Let chosen custom attributes from Lab 4.1 reach the page views
sed -i '/^  log_level: info/a\  browser_monitoring:\n    auto_instrument: true\n
attributes:\n      enabled: true\n      include: search.*' \
  ~/nr-lab/agent/petclinic/newrelic.yml

cd ~/nr-lab && pcbuild
curl -s http://localhost:8090/vets.html | grep -o 'NREUM[^;]\{0,30\}' | head -2
```

Lab 4.2 · Step 2 · Browser agent in the pages — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab/petclinic-src$ sed -i '/<meta name="viewport"/a\  <th:block
                                       th:utext="${newrelic.browserHeader}"></th:block>'
                                       src/main/resources/templates/fragments/layout.html
trainee@nr-lab:~/nr-lab/petclinic-src$ sed -i 's|</body>|<script> … fetch("/vets") …
                                       petclinicUi.highlightVets() … </script>\n</body>|'
                                       src/main/resources/templates/vets/vetList.html
trainee@nr-lab:~/nr-lab/petclinic-src$ sed -i '/^  log_level: info/a\  browser_monitoring: …'
                                       ~/nr-lab/agent/petclinic/newrelic.yml
trainee@nr-lab:~/nr-lab/petclinic-src$ cd ~/nr-lab && pcbuild
building against newrelic-api 8.25.0
[+] Restarting 1/1
 ✔ Container petclinic  Started
petclinic: HTTP 200
trainee@nr-lab:~/nr-lab/petclinic-src$ curl -s http://localhost:8090/vets.html | grep -o
                                       'NREUM[^;]\{0,30\}' | head -2
NREUM||(NREUM={})
NREUM.init={distributed_tracing:{enabled:t
the browser agent script is now in the <head> of every PetClinic page
```

Expected output — PetClinic rebuilt; the rendered page now contains the browser agent script

The script is not in the page? The agent receives the browser settings when it connects: check that Step 1 is done, restart PetClinic once more, and look for `browser` in `~/nr-`

`lab/agent/petclinic/logs/newrelic_agent.log`.

Step 3 — Real browsers

The browser agent is JavaScript, so `curl` traffic creates no page views. Run headless Chrome browsers with Playwright, each visit with a different device profile:

```
mkdir -p ~/nr-lab/rum
cat > ~/nr-lab/rum/pcbrowse.mjs <<'EOF'
import { chromium, devices } from 'playwright';

const BASE = process.env.BASE || 'http://localhost:8090';
const profiles = ['Desktop Chrome', 'Desktop Edge', 'Pixel 7', 'iPhone 13', 'Galaxy S9+'];
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const browser = await chromium.launch();

while (true) {
  const name = profiles[Math.floor(Math.random() * profiles.length)];
  const context = await browser.newContext({ ...devices[name] });
  const page = await context.newPage();
  try {
    await page.goto(BASE + '/');
    await page.goto(BASE + '/owners/find');
    await page.fill('#lastName', 'Last' + Math.floor(Math.random() * 50000));
    await page.click('#search-owner-form button[type="submit"]');
    await page.waitForLoadState('load');
    const first = page.locator('#owners a').first();
    if (await first.count()) { await first.click(); await page.waitForLoadState('load'); }
    await page.goto(BASE + '/vets.html');
    await sleep(3000);                                  // let the browser agent send its
data
    console.log(new Date().toISOString(), 'visit done:', name);
  } catch (e) {
    console.log(new Date().toISOString(), 'visit failed:', name, e.message);
  }
  await context.close();
  await sleep(4000);
}
EOF
docker run -d --name pc-browser --restart unless-stopped --network host --ipc=host \
  -v "$HOME/nr-lab/rum":/rum -w /rum mcr.microsoft.com/playwright:v1.48.0-noble \
  sh -c "npm init -y > /dev/null && npm install --no-audit --no-fund playwright@1.48.0 >
/dev/null && node pcbrowse.mjs"
sleep 90; docker logs --tail 5 pc-browser
```

Lab 4.2 · Step 3 · Headless browsers — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ docker run -d --name pc-browser --restart unless-stopped --network host
                          --ipc=host -v "$HOME/nr-lab/rum":/rum -w /rum
                         mcr.microsoft.com/playwright:v1.48.0-noble sh -c "npm init -y >
                         /dev/null && npm install --no-audit --no-fund playwright@1.48.0 >
                         /dev/null && node pcbrowse.mjs"
Unable to find image 'mcr.microsoft.com/playwright:v1.48.0-noble' locally
v1.48.0-noble: Pulling from playwright
Status: Downloaded newer image for mcr.microsoft.com/playwright:v1.48.0-noble
8d41c07e9b2a51f3c6e0a7d9b4f2e1c0a9b8d7e6f5a4b3c2d1e0f9a8b7c6d5e4
trainee@nr-lab:~/nr-lab$ sleep 90; docker logs --tail 5 pc-browser
2026-10-29T12:04:31.118Z visit done: Pixel 7
2026-10-29T12:04:42.906Z visit done: Desktop Chrome
2026-10-29T12:04:54.377Z visit done: iPhone 13
2026-10-29T12:05:05.815Z visit done: Galaxy S9+
2026-10-29T12:05:17.240Z visit done: Desktop Edge
```

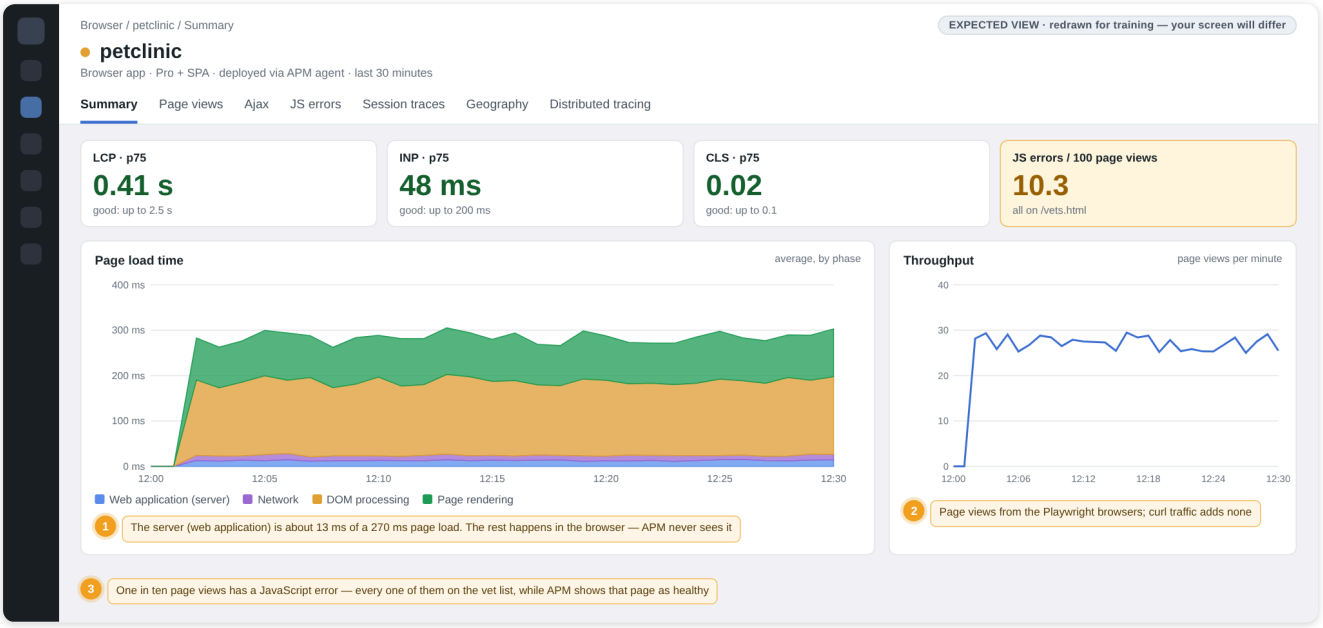
Expected output — the Playwright container visiting PetClinic with different device profiles

Also open `http://<VM-IP>:8090/` in your own browser and click through a few pages: your visits add a second location and a real device.

Step 4 — The browser summary

After 10 minutes, open **Browser** → **petclinic** → **Summary**:

- **Page load time**, split into *web application* (the server), *network*, *DOM processing* and *page rendering*.
- **Core Web Vitals**: LCP, INP and CLS at the 75th percentile.
- **Throughput** (page views per minute), **JavaScript errors** and **Ajax** response time.



Expected view — browser summary for petclinic: page load split into phases, Core Web Vitals, JS error rate and Ajax

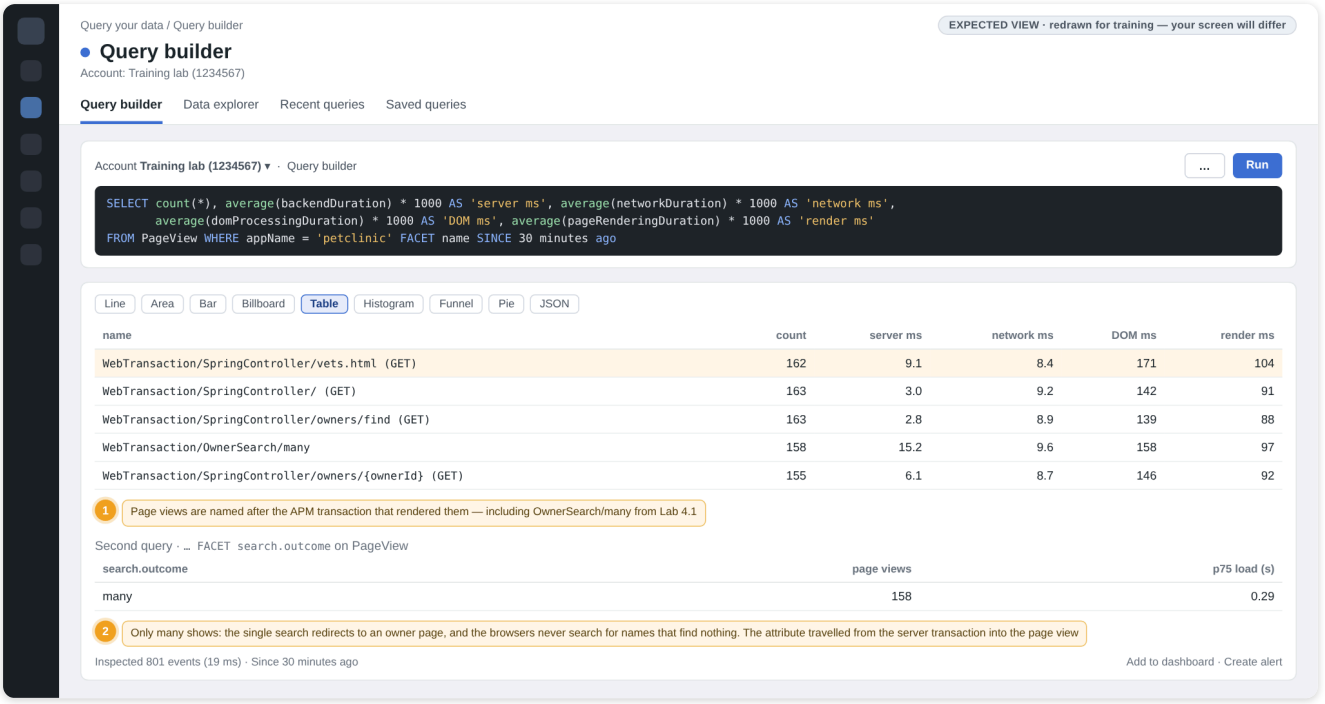
Question 1. The *web application* part of the page load is a few milliseconds, but the total is several hundred. Where does the rest go, and would APM ever show it?

Step 5 — Page views and the custom attributes

Open **Page views**, sort by **Most time consuming**. Each page is named after the transaction that rendered it — including `OwnerSearch/many` from Lab 4.1.

```
SELECT count(*), average(backendDuration) * 1000 AS 'server ms', average(networkDuration) *
1000 AS 'network ms',
       average(domProcessingDuration) * 1000 AS 'DOM ms', average(pageRenderingDuration) *
1000 AS 'render ms'
FROM PageView WHERE appName = 'petclinic' FACET name SINCE 30 minutes ago

SELECT count(*), percentile(duration, 75) FROM PageView
WHERE appName = 'petclinic' AND search.outcome IS NOT NULL FACET search.outcome SINCE 30
minutes ago
```



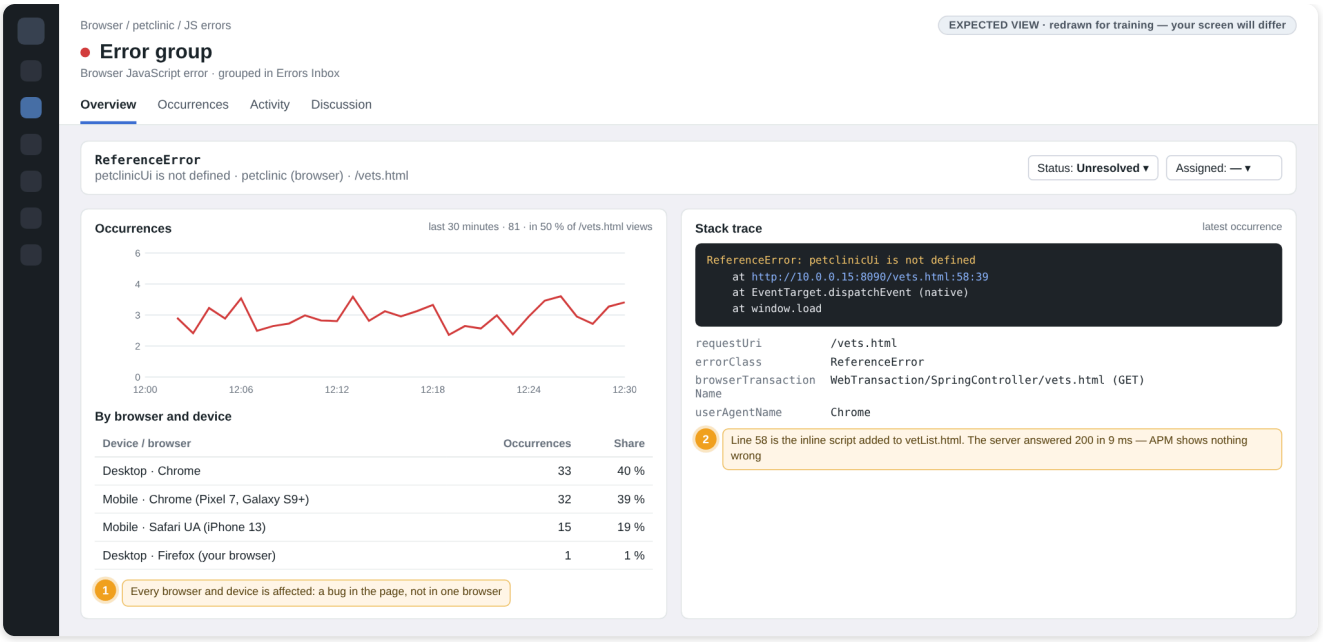
Expected view — page load phases per page, and the search.outcome attribute carried from the APM transaction into the page views

The second query works because `browser_monitoring.attributes` passes the `search.*` attributes from the server transaction into the page view.

Step 6 — JavaScript errors

Open **Browser** → **petclinic** → **JS errors** (also in **Errors Inbox** with the browser app in scope):

- `ReferenceError: petclinicUi is not defined` on `/vets.html`, in about half of the page views.
- The stack trace points at the inline script; the breakdown shows every browser and device profile is affected.
- APM shows `/vets.html` as **successful** — the server cannot see this error.



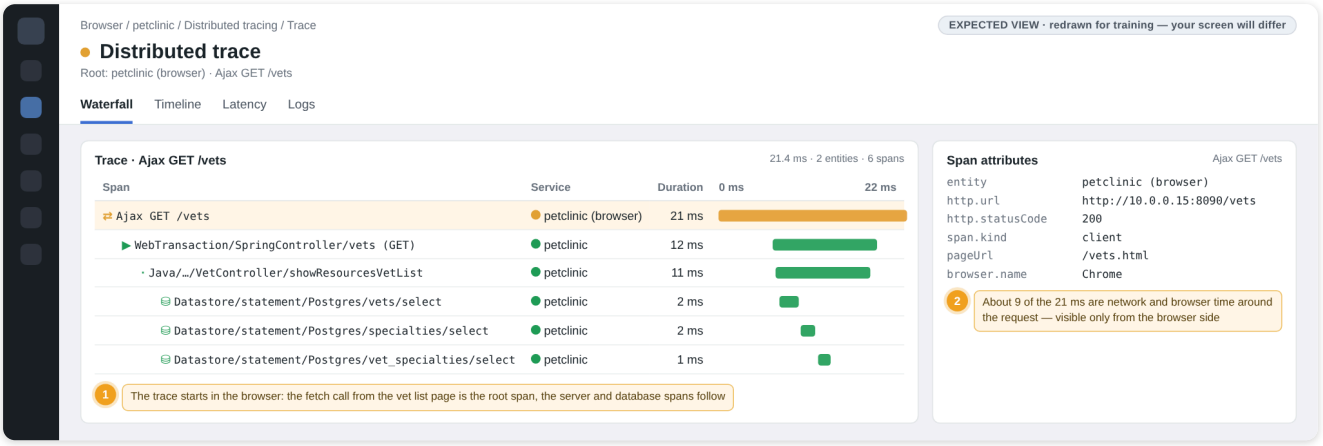
Expected view — the JavaScript error group: message, page, occurrences across browsers, and the stack trace

```
SELECT count(*) FROM JavaScriptError WHERE appName = 'petclinic'
FACET errorClass, errorMessage, requestUri, deviceType SINCE 30 minutes ago
```

Step 7 — From the browser into the backend: Ajax and browser traces

Open **Ajax**: the `GET /vets` call from the vet list page, with its response time and status. Then **Distributed tracing**, filter on `petclinic` browser spans, and open a trace that starts in the browser:

- The **browser span** (the Ajax request) is the root.
- Below it, the **petclinic** server span `/vets` and its **Postgres vets select** spans.



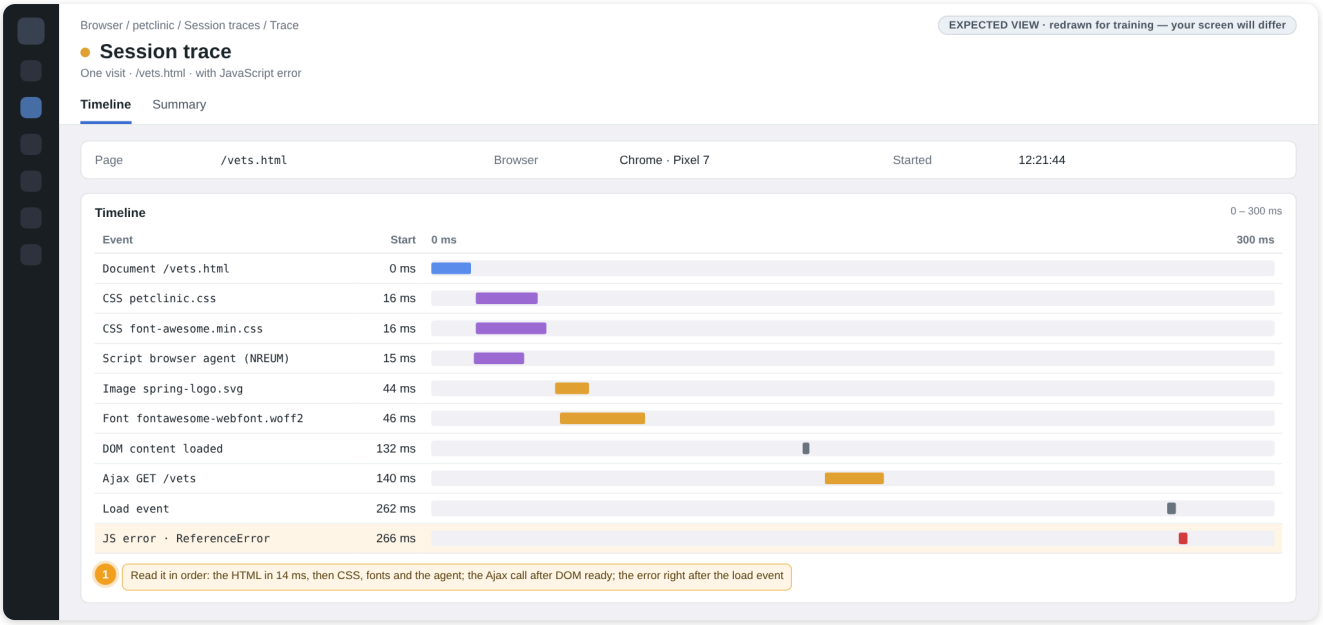
Expected view — a distributed trace that starts with the browser Ajax call and continues through PetClinic into PostgreSQL

```
SELECT count(*), average(timeToLoadEventStart) * 1000 AS 'ms' FROM AjaxRequest
WHERE appName = 'petclinic' FACET requestUrl, httpResponseCode SINCE 30 minutes ago
```

Step 8 — Session traces

Open **Session traces** (or **Session replay**, if enabled for the account) and pick a visit to `/vets.html` that had the error:

- The timeline shows the HTML document, CSS, JavaScript and font downloads, the `fetch` call to `/vets`, the load event — and the JavaScript error marked at the moment it happened.



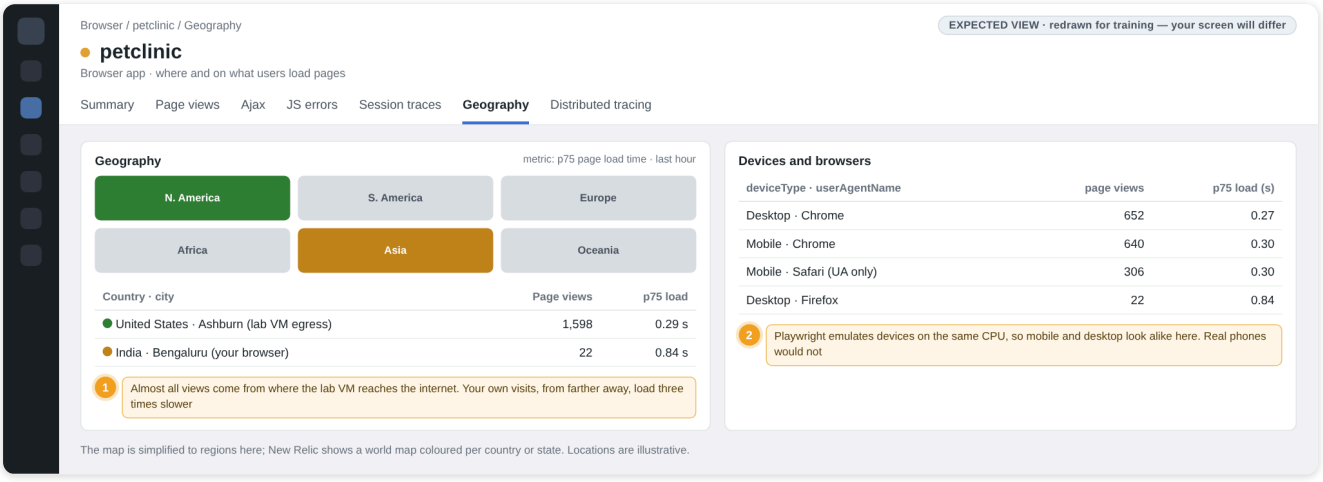
Expected view — session trace timeline of one vet list visit: resources, the Ajax call, the load event and the JS error

Step 9 — Geography and devices

Open **Geography**: the map shows where page views come from (the lab VM's internet egress, and your own location if you browsed). Change the metric to *page load time* and *LCP*.

```
SELECT count(*), percentile(duration, 75) AS 'p75 load (s)' FROM PageView
WHERE appName = 'petclinic' FACET countryCode, city SINCE 1 hour ago

SELECT count(*), percentile(duration, 75) FROM PageView
WHERE appName = 'petclinic' FACET deviceType, userAgentName SINCE 1 hour ago
```



Expected view — geography: page views and p75 load time by country and city, and the device breakdown

Check your work

- ☐ `PageView`, `PageViewTiming`, `JavaScriptError` and `AjaxRequest` events arrive for `petclinic`.
- ☐ You can show a trace that starts in the browser and ends in PostgreSQL.

- ☐ You can explain the `ReferenceError` from the browser data alone, and why APM shows `/vets.html` as healthy.

Answer. 1 — Network transfer, downloading CSS, JavaScript and fonts, parsing the HTML, running scripts and painting the page all happen after the server has answered. APM ends when the response leaves the server, so it never shows that time; only the browser agent measures it.

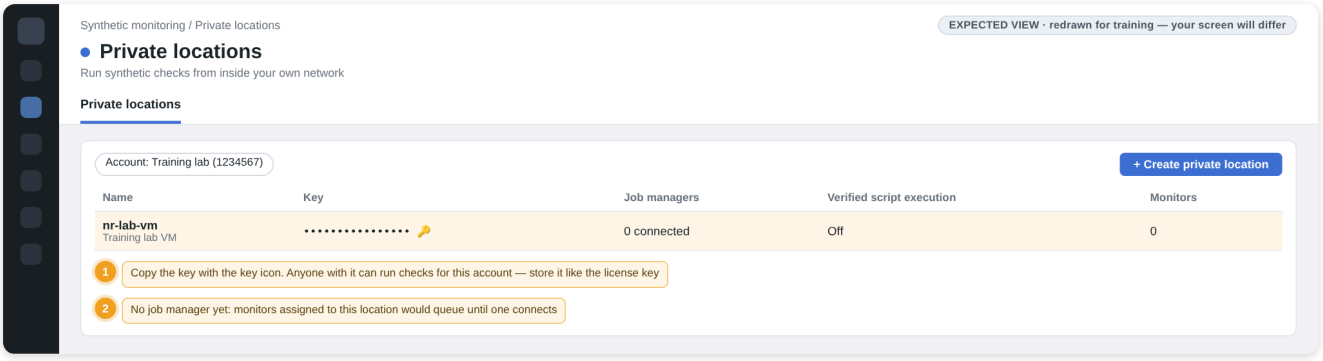
Lab 4.3 — Synthetic monitoring from a private location, with SLA and alerts

Goal: check PetClinic and easyTravel from the outside every minute — availability, an API contract and a full user journey — from a private location on the lab VM; read the results, the linked backend traces and the SLA report; alert on failures and on the location going silent; then rehearse an outage.

Time: 75 minutes. **Module:** 12.

Step 1 — Create the private location

1. **Synthetic monitoring → Private locations → Create private location.**
2. Name `nr-lab-vm`, description *Training lab VM*. Leave *verified script execution* off for the lab.
3. **Save**, then copy the **private location key** (the key icon).



Expected view — the new private location and its key; no job manager connected yet

Store the key like the license key — it lets anyone who has it run jobs for your account:

```
read -rsp "Private location key: " PLK; echo
echo "NR_PRIVATE_LOCATION_KEY=$PLK" >> ~/nr-lab/.env && unset PLK
chmod 600 ~/nr-lab/.env
```

Step 2 — Run the synthetics job manager

```
set -a; source ~/nr-lab/.env; set +a
docker run -d --name synthetics-job-manager --restart unless-stopped \
  -e PRIVATE_LOCATION_KEY="$NR_PRIVATE_LOCATION_KEY" \
  -e HEAVYWEIGHT_WORKERS=1 \
  -v /var/run/docker.sock:/var/run/docker.sock:rw \
  newrelic/synthetics-job-manager:latest
sleep 120; docker logs --tail 8 synthetics-job-manager
hostname -I | awk '{print "lab VM IP: " $1}'
```

Lab 4.3 · Step 2 · Synthetics job manager — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ set -a; source ~/nr-lab/.env; set +a
trainee@nr-lab:~/nr-lab$ docker run -d --name synthetics-job-manager --restart unless-stopped -e
                          PRIVATE_LOCATION_KEY="$NR_PRIVATE_LOCATION_KEY" -e
                         HEAVYWEIGHT_WORKERS=1 -v
                         /var/run/docker.sock:/var/run/docker.sock:rw
                         newrelic/synthetics-job-manager:latest
Status: Downloaded newer image for newrelic/synthetics-job-manager:latest
c41e8a07d2b95f63a1e0c7d8b9a2f4e5d6c7b8a9f0e1d2c3b4a5f6e7d8c9b0a1
trainee@nr-lab:~/nr-lab$ sleep 120; docker logs --tail 8 synthetics-job-manager
INFO  Synthetics Job Manager starting · private location nr-lab-vm
INFO  Heavyweight workers: 1
INFO  Pulling runtime image newrelic/synthetics-ping-runtime
INFO  Pulling runtime image newrelic/synthetics-node-api-runtime
INFO  Pulling runtime image newrelic/synthetics-node-browser-runtime
INFO  Connected to New Relic · polling for jobs
(log wording varies by job manager release)
trainee@nr-lab:~/nr-lab$ hostname -I | awk '{print "lab VM IP: " $1}'
lab VM IP: 10.0.0.15
```

Expected output — the job manager connected to the private location and pulling its runtime images; the VM's IP for the monitors

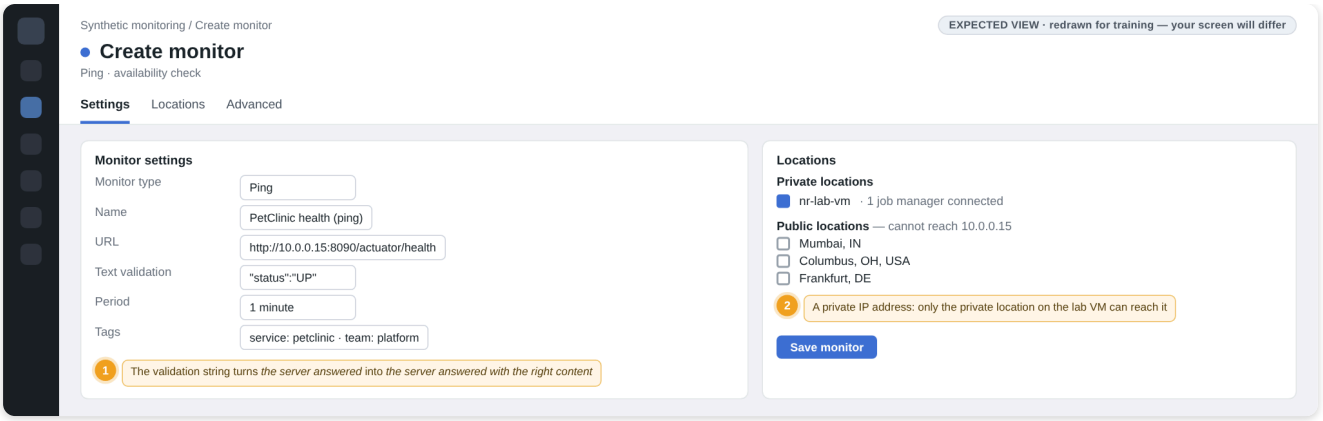
- `HEAVYWEIGHT_WORKERS=1` keeps the job manager to one browser or API check at a time, which fits the lab VM; the default is the number of CPUs minus one.
- EU-region accounts also set `-e HORDE_API_ENDPOINT=https://synthetics-horde.eu01.nr-data.net/`.
- Checks run in separate containers, so monitors must use the **VM's IP**, not `localhost`. Below, replace `10.0.0.15` with your VM's IP.

The private location page now shows the job manager as connected.

Step 3 — Ping monitors: are the sites up?

Synthetic monitoring → **Create monitor** → **Ping**, twice:

Name	URL	Validation text	Period	Location
`easyTravel home (ping)`	`http://10.0.0.15/`	`easyTravel`	1 minute	nr-lab-vm
`PetClinic health (ping)`	`http://10.0.0.15:8090/actuator/health`	`"status":"UP"`	1 minute	nr-lab-vm



Expected view — creating the PetClinic health ping monitor with a validation string and the private location

Validation text makes the check fail when the server answers but serves the wrong content — an error page with HTTP 200 still fails.

Step 4 — A scripted API test

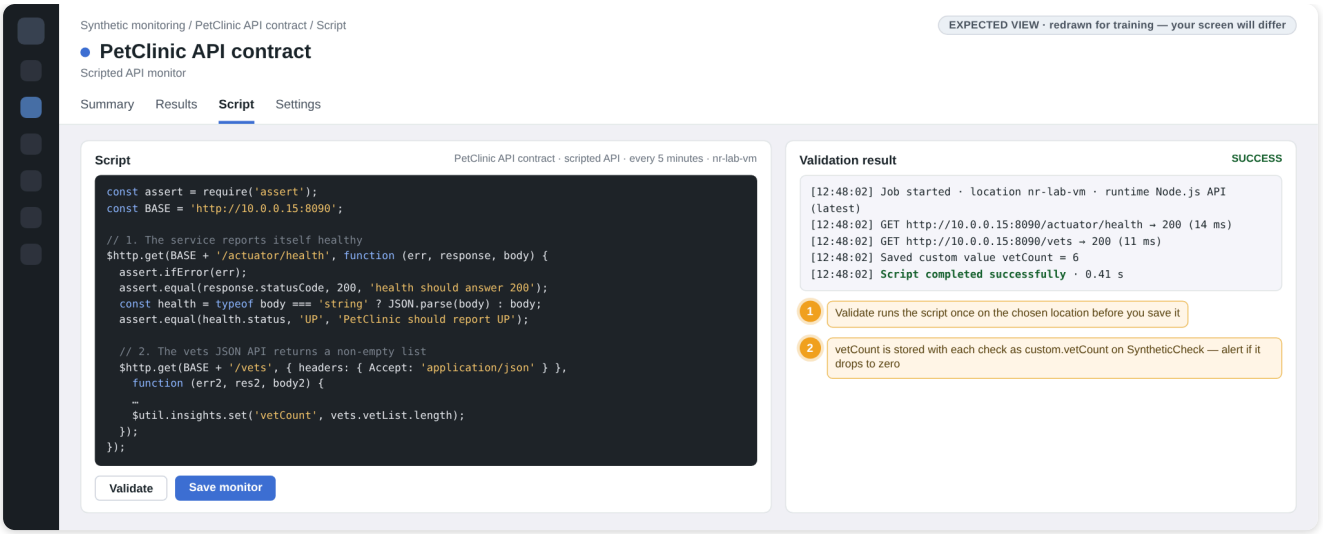
Create monitor → **Scripted API**, name `PetClinic API contract`, period 5 minutes, location `nr-lab-vm`, the newest Node.js runtime:

```
const assert = require('assert');
const BASE = 'http://10.0.0.15:8090';

// 1. The service reports itself healthy
$http.get(BASE + '/actuator/health', function (err, response, body) {
  assert.ifError(err);
  assert.equal(response.statusCode, 200, 'health should answer 200');
  const health = typeof body === 'string' ? JSON.parse(body) : body;
  assert.equal(health.status, 'UP', 'PetClinic should report UP');

  // 2. The vets JSON API returns a non-empty list
  $http.get(BASE + '/vets', { headers: { Accept: 'application/json' } }, function (err2,
res2, body2) {
    assert.ifError(err2);
    assert.equal(res2.statusCode, 200, '/vets should answer 200');
    const vets = typeof body2 === 'string' ? JSON.parse(body2) : body2;
    assert.ok(vets.vetList.length > 0, 'the vet list should not be empty');
    $util.insights.set('vetCount', vets.vetList.length);
  });
});
```

Click **Validate**: the script runs once on the private location and shows the log and result. Save the monitor.



Expected view — the scripted API monitor: the script, a successful validation run and the saved custom value

`$util.insights.set` saves a value with the check result (`custom.vetCount` on `SyntheticCheck`), so you can chart and alert on it.

Step 5 — A scripted browser journey

Create monitor → **Scripted browser**, name `PetClinic find owner journey`, period 5 minutes, location `nr-lab-vm`, the latest Chrome runtime:

```
const assert = require('assert');
const By = $selenium.By, until = $selenium.until;
const BASE = 'http://10.0.0.15:8090';

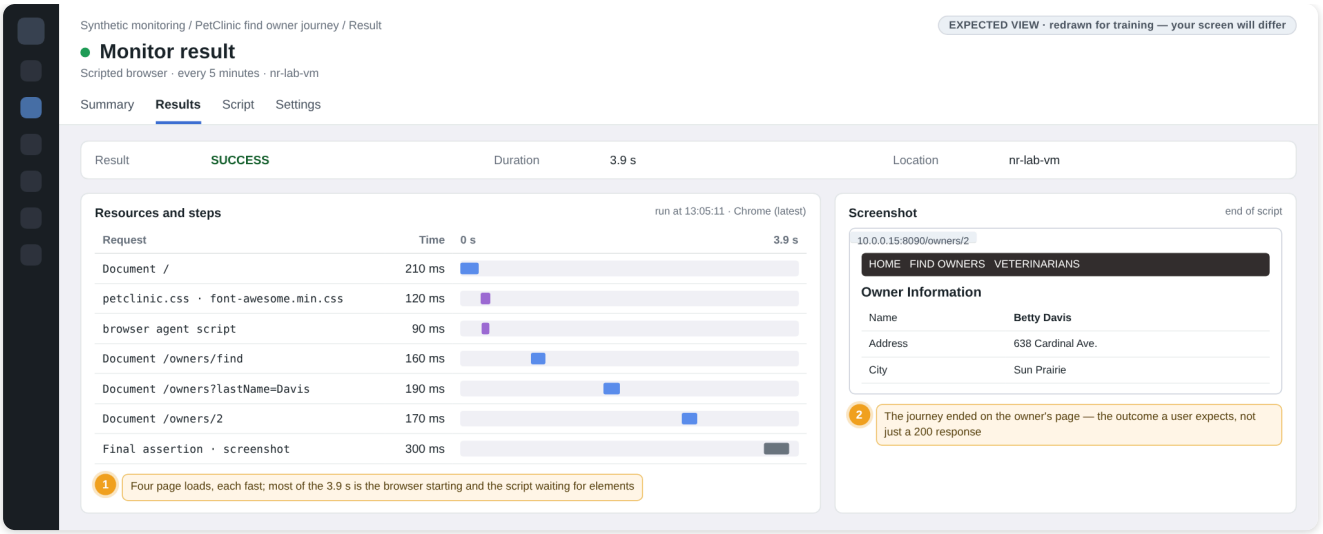
// 1. Home page, then the find-owners form
await $webDriver.get(BASE + '/');
await $webDriver.findElement(By.css('a[href="/owners/find"]')).click();
const box = await $webDriver.wait(until.elementLocated(By.id('lastName')), 10000);

// 2. Search for a known owner name
await box.sendKeys('Davis');
await $webDriver.findElement(By.css('#search-owner-form button[type="submit"]')).click();
const table = await $webDriver.wait(until.elementLocated(By.id('owners')), 10000);
assert.ok((await table.getText()).includes('Davis'), 'the results should list owners named
Davis');

// 3. Open one owner and check the details page
await $webDriver.findElement(By.linkText('Betty Davis')).click();
await $webDriver.wait(until.elementLocated(By.xpath("//h2[contains(., 'Owner
Information')]")), 10000);
```

Validate, then save. Open the monitor's **Results** and one run:

- The **result** and **duration**, the **resource waterfall** (HTML, CSS, scripts, the browser agent itself) and a **screenshot** at the end.
- The page loads appear in APM as transactions tagged by synthetics.



Expected view — one journey run: steps, duration, the resource waterfall and the final screenshot

```
SELECT latest(result), latest(duration) / 1000 AS 'last run (s)' FROM SyntheticCheck
FACET monitorName, locationLabel SINCE 30 minutes ago

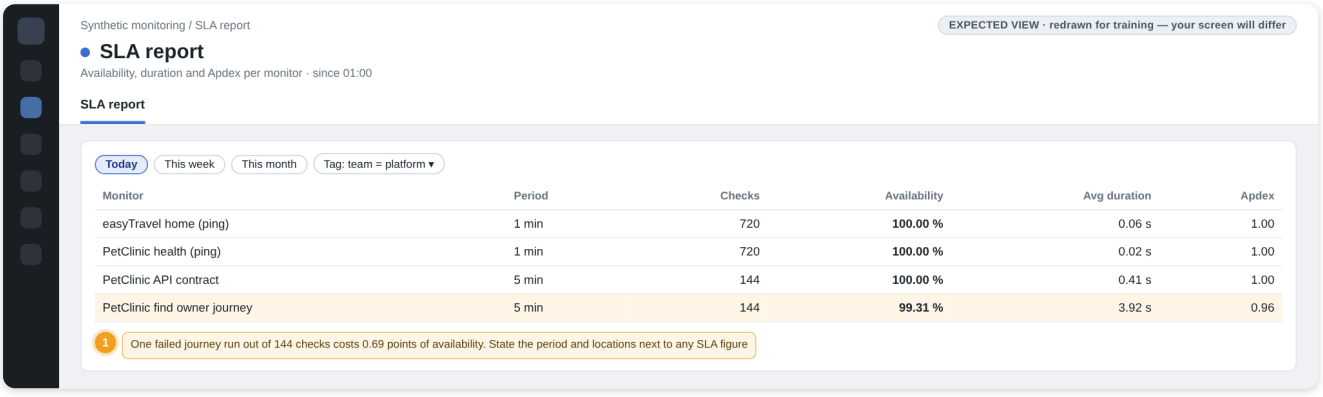
SELECT count(*), average(duration) * 1000 AS 'avg ms' FROM Transaction
WHERE appName = 'petclinic' AND nr.syntheticsMonitorId IS NOT NULL FACET name SINCE 30
minutes ago
```

Question 1. The journey monitor loads PetClinic pages in Chrome, so the browser agent runs. How do you keep synthetic visits out of business reports such as "page views per day"?

Step 6 — SLA report

After 30 minutes, open **Synthetic monitoring → SLA report** (or a monitor's *SLA* view): availability and Apdex per monitor, per day.

```
SELECT percentage(count(*), WHERE result = 'SUCCESS') AS 'availability %',
       percentile(duration, 95) / 1000 AS 'p95 (s)', count(*) AS 'checks'
FROM SyntheticCheck FACET monitorName SINCE 1 day ago
```



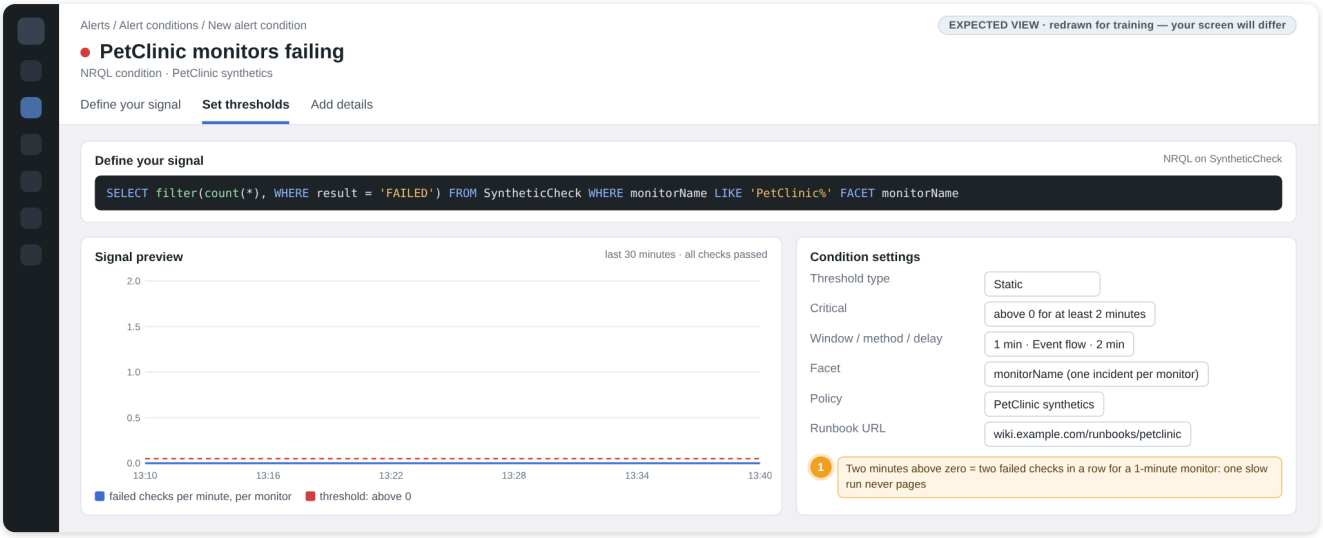
Expected view — SLA report: availability, checks and duration per monitor for the day

Note the `checks` column: the 1-minute ping monitors run 1,440 checks a day, the 5-minute journey 288. One failed journey check moves its daily availability by about 0.35 points.

Step 7 — Alert on journey failures and on a silent location

Create a policy `PetClinic synthetics` (issue preference **One issue per policy**) with two NRQL conditions:

Name	Query	Threshold
`PetClinic monitors failing`	`SELECT filter(count(*), WHERE result = 'FAILED') FROM SyntheticCheck WHERE monitorName LIKE 'PetClinic%' FACET monitorName`	Critical above 0 for at least 2 minutes (window 1 minute, event flow, delay 2 minutes)
`Private location silent`	`SELECT count(*) FROM SyntheticCheck WHERE locationLabel = 'nr-lab-vm'`	Critical below 1 for at least 5 minutes; **signal loss** after 5 minutes → open new incident



Expected view — the failing-monitors condition: NRQL on SyntheticCheck faceted by monitor, two minutes above zero

Two minutes above zero means at least two failed checks in a row for the 1-minute monitors, so a single slow run does not page anyone. The second condition catches the job manager being stopped — without it, a dead private location looks like "no failures".

Step 8 — Rehearse an outage

```
date +%H:%M; pcdc stop petclinic
sleep 300
docker logs --tail 4 synthetics-job-manager
pcdc start petclinic && sleep 90; date +%H:%M
curl -s -o /dev/null -w "petclinic: HTTP %{http_code}\n"
http://localhost:8090/actuator/health
```

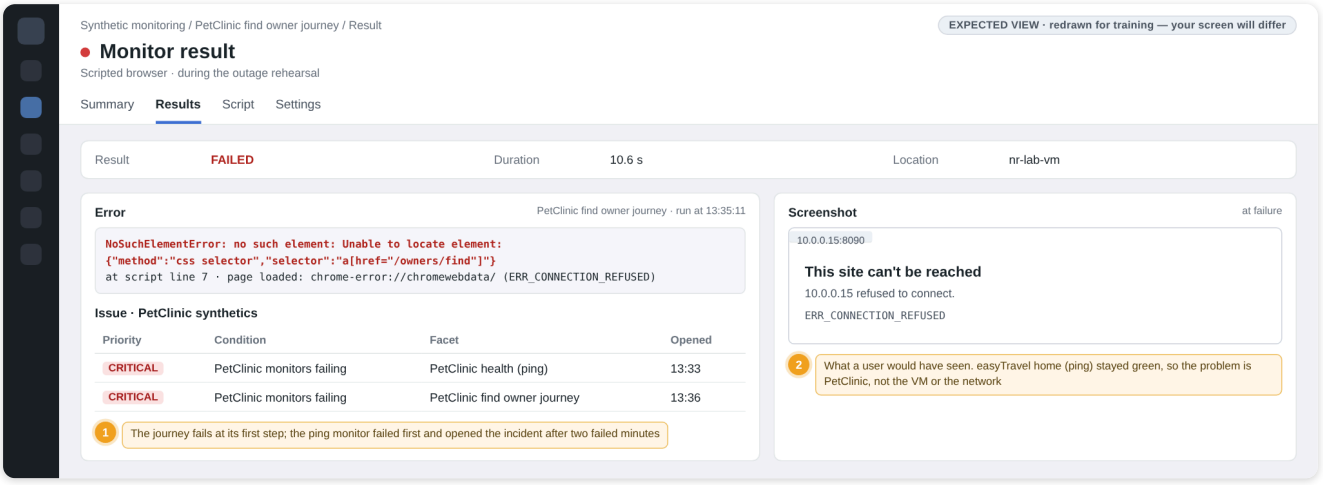
Lab 4.3 · Step 8 · Outage rehearsal — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ date +%H:%M; pcdc stop petclinic
13:31
[+] Stopping 1/1
 ✔ Container petclinic  Stopped
trainee@nr-lab:~/nr-lab$ sleep 300
trainee@nr-lab:~/nr-lab$ docker logs --tail 4 synthetics-job-manager
INFO  Job completed · PetClinic health (ping) · FAILED
INFO  Job completed · easyTravel home (ping) · SUCCESS
INFO  Job completed · PetClinic find owner journey · FAILED
INFO  Job completed · PetClinic health (ping) · FAILED
trainee@nr-lab:~/nr-lab$ pcdc start petclinic && sleep 90; date +%H:%M
[+] Running 1/1
 ✔ Container petclinic  Started
13:38
trainee@nr-lab:~/nr-lab$ curl -s -o /dev/null -w "petclinic: HTTP %{http_code}\n"
                           http://localhost:8090/actuator/health
petclinic: HTTP 200
the incident closes once the monitors pass again
```

Expected output — PetClinic stopped for five minutes, then started again

While PetClinic is down:

- The PetClinic ping and API monitors fail with connection errors; the journey fails at its first step with a screenshot of the browser error page; `easyTravel home (ping)` stays green.
- `PetClinic monitors failing` opens an incident after about two failed minutes.
- Browser data from the Playwright container stops; APM shows no throughput.



Expected view — the failed journey run: the error, the failing step and the screenshot; the open incident

After the restart, the checks pass again and the incident closes. Record the outage in your notes: start, first failed check, incident opened, recovery.

Step 9 — Clean up costs

Synthetic checks count toward your account's synthetics usage. After the course, set the monitors to longer periods or disable them, and stop the job manager when the VM is not in use: `docker stop synthetics-job-manager pc-browser`.

Check your work

- ☐ The private location shows a connected job manager; all four monitors have successful runs.
- ☐ You can show the backend transactions caused by the journey monitor.
- ☐ The outage opened an incident from the failing monitors, and it closed after recovery.
- ☐ The private location key is only in `~/nr-lab/.env` (mode 600).

Answer. 1 — Synthetic traffic is marked: filter it out, for example with `WHERE nr.syntheticsMonitorId IS NULL` on transactions, or by excluding the user agent of synthetic checks in browser queries. Many teams also send synthetic traffic to a test account or test data so business events are never polluted.

Appendix — Reset, troubleshooting and end-of-course checklist

Reset everything

```
pkill -f pcload.sh || true
docker stop pc-browser synthetics-job-manager
pcdc ps --format "table {{.Name}}\t{{.Status}}"
docker ps --format "table {{.Names}}\t{{.Status}}"
```

To undo today's code changes: `cd ~/nr-lab/petclinic-src && git checkout -- . && git clean -fd` and run `pcbuild` again. To remove the containers: `docker rm -f pc-browser synthetics-job-manager`, then delete the private location and the monitors in New Relic.

Troubleshooting

Symptom	Check	Fix
No custom segment after Step 2	`ls ~/nr-lab/agent/petclinic/extensions`	The file must be in `extensions/` next to `newrelic.jar`; class and method names exact; restart PetClinic
Build fails: `newrelic-api` not found	`pcbuild` output	`NR_VERSION` empty or not a released version — print it; it must match the agent jar's version
Build fails on formatting or checkstyle	Build output	Use `pcbuild`, which skips those checks
No `OwnerSearch` events	`grep OwnerSearchTelemetry .../OwnerController.java`	The `sed` insert did not match; add the call by hand after the `findPaginatedForOwnersLastName` line
Transaction still named `owners (GET)`	A trace of the search	The new jar is not running: `pcdc restart petclinic`; check the jar's date
No `NREUM` in the page source	`grep -i browser ~/nr-lab/agent/petclinic/logs/newrelic_agent.log`	Browser monitoring not enabled for the APM app (Step 1), or the agent has not reconnected: restart PetClinic
Page source has the script, no page views	`docker logs --tail 20 pc-browser`	Browsers cannot reach the internet (`bam.nr-data.net`); check the VM's outbound HTTPS
`search.outcome` missing on PageView	Agent config	`browser_monitoring.attributes.enabled: true` and `include: search.*` under `common:`; restart
Job manager not connected	`docker logs --tail 50 synthetics-job-manager`	Wrong key or region (EU needs `HORDE_API_ENDPOINT`); outbound HTTPS blocked; Docker socket not mounted
Browser and API checks queue or time out	Private location page: pending checks	Not enough memory or CPU for the job manager; keep `HEAVYWEIGHT_WORKERS=1`; lengthen monitor periods
Monitors fail with connection refused	Monitor URL	`localhost` used instead of the VM's IP; a host firewall blocking Docker networks

End-of-course checklist

- ☐ All load scripts and headless browsers stopped; the job manager stopped or removed if the VM is returned.
- ☐ Monitors disabled or deleted, the private location deleted, and the `PetClinic synthetics` policy disabled in shared accounts.
- ☐ The private location key, license key and user key are deleted from the VM or rotated.
- ☐ Your Day 1–4 queries, runbooks, dashboard JSON and code changes are saved outside the lab VM.

Sources

- New Relic — Guide to using the Java agent API
- New Relic — Java XML instrumentation examples
- New Relic — Instrument browser monitoring with the Java agent API
- New Relic — Java agent browser monitoring settings
- New Relic — Install the synthetics job manager
- New Relic — Job manager configuration
- New Relic — Write synthetic API tests
- New Relic — Introduction to scripted browser monitors
- Playwright — Docker images